

Hospital Management System

Design Specification Document

v 1.0

Prepared by

Rawan Muhammad

Doha Nour El-Din

Salma El-Hawary

Habeba Saad

Yara Abdelkader

Organization:

Software Engineering Course –The American University in Cairo

Date Created:

10/10/2025

Table of Contents

Table of Contents.....	1
1. Introduction.....	4
1.1 Purpose.....	4
1.1.1 Relation to Requirements.....	4
1.2 Scope.....	4
1.3 Definitions, Acronyms, and Abbreviations.....	5
1.4 References.....	6
1.5 Overview.....	6
2. System Architecture.....	7
2.1 Design Goals & Constraints.....	7
2.2 Architectural Style.....	7
2.3 Module Decomposition.....	8
2.4 Interaction Between Modules.....	9
2.4.1 Authentication module.....	9
2.4.2 Booking appointment module.....	9
2.4.3 Billing system module.....	11
Fig 5. Sequence diagram of billing system.....	11
2.4.4 Communication and support module.....	11
Fig 6. Sequence diagram for Communication and support.....	12
2.4.5 Shift scheduling system module.....	12
Fig 7. Sequence diagram for shift scheduling system.....	13
2.4.6 Medical record module.....	13
Fig 8. Sequence diagram of medical record.....	13
Fig 9. Sequence diagram for Administration System.....	14
3. Data Design.....	15
3.1 Data Model / ER Diagram.....	15
Fig 10. Class diagram for HMS.....	15
3.2 Data Dictionary.....	16
3.2.1 User.....	16
3.2.2 Admin.....	16
3.2.3 SupportGroup.....	17
3.2.4 Patient.....	17
3.2.5 Doctor.....	18
3.2.6 Nurse.....	18
3.2.7 Receptionist.....	19
3.2.8 Clinic.....	19
3.2.9 Appointment.....	20
3.2.9 Payment.....	21
3.2.10 Shift.....	21

3.2.11 Message.....	22
3.2.12 MedicalRecord.....	22
3.2.13 Prescription.....	23
3.2.14 UI.....	23
3.3 Persistence / Storage Strategy.....	24
4.1 User Interfaces.....	25
Fig 11. UI for login, sign_up, and patient dashboard.....	25
Fig 12. Booking appointments, canceling appointments, and confirmation messages.....	26
Fig 13. Profile page, booked appointments screen in patients account, and doctors' signup screen..	27
Fig 14. UI for adding new slots by doctors, the doctor's dashboard, and the success message after adding a slot.....	28
4.2 API / Software Interfaces.....	29
4.2.1 Authentication and User Management Interfaces.....	29
4.2.2 Patient Profile.....	29
4.2.3 Clinics and Doctors Listing.....	29
4.2.4 Appointment Management.....	30
4.2.5 Error Handling.....	30
4.3 Hardware Interfaces (if applicable).....	30
4.4 Communications / Network Interfaces.....	31
4.4.1 Protocols and Message Formats.....	32
4.4.2 Security Mechanisms.....	32
4.4.3 Bandwidth Considerations.....	32
4.4.4 Error Handling and Reliability.....	32
5. Component Design & Algorithms.....	33
5.1 Component Descriptions.....	33
5.1.1 User & Authentication Component.....	33
5.2 Key Algorithms.....	34
5.2.1 Appointment Conflict Detection.....	34
5.2.2 Appointment Rescheduling Logic.....	34
5.2.3 Shift Assignment and Overlap Prevention.....	35
5.2.4 Role-Based Access Control Evaluation.....	35
5.2.5 Doctor Availability Validation.....	35
6. Security & Safety.....	36
6.1 Authentication & Authorization.....	36
6.2 Data Privacy & Encryption.....	36
6.3 Audit Logging & Monitoring.....	37
6.4 Failure Modes & Recovery.....	37
7. Performance, Scalability & Availability.....	38
7.1 Performance Requirements.....	38
7.2 Scalability.....	38

7.3 Availability & Fault Tolerance.....	38
8. Design Risks and Mitigations.....	39
8.1 Third-Party Backend Dependency (Firebase).....	39
9. Testing & Verification Strategy.....	41
9.1 Unit and Integration Testing.....	41
Unit Testing.....	41
Integration Testing.....	41
9.2 Performance / Load Testing.....	41
9.3 Security Testing.....	42
9.4 User Acceptance / UI Testing.....	42
10. Appendices.....	43

1. Introduction

1.1 Purpose

The purpose of this document is to provide a detailed description of how the hospital management system (HMS) will be designed and implemented to meet the requirements specified in the Software Requirements Specification (SRS). It translates the functional and non functional requirements into technical points that guide the developer, designers, and testers (us) during the system development's cycle. It serves as a reference for verifying that the design meets all specified requirements needed.

This document is intended for:

- Developer
- Tester
- Project Manager
- Stakeholders and Clients: to know their roles, responsibilities, and be sure that the system is fully secured and easy to use (i.e. follows the functional and non-functional requirements "FURPS" well)

1.1.1 Relation to Requirements

This document is directly derived from the Software Requirements Specification (SRS), which outlines what the system should do. The Design Specification outlines how the system will achieve those requirements by detailing the architecture, modules, data design, and interaction flows. It makes sure that each design decision aligns with the stated functional and non-functional requirements of the Hospital Management System (HMS).

1.2 Scope

The scope of the Carevo Hospital Management System (HMS) in this design focuses on the core patient-facing and basic staff-support workflows needed to manage appointments and related information in a hospital setting. It includes user registration and authentication for patients and staff, patient profile management, viewing clinics by category, listing doctors and their available time slots, and the complete appointment lifecycle: booking, rescheduling, cancellation, and viewing past and upcoming visits. The system also covers basic messaging and support interactions, as well as shift and schedule views for nurses and doctors at the design level, aligned with the documented user stories and class model.

Out of scope for this iteration are advanced inpatient modules (such as bed management and full EMR workflows), complex billing and insurance processing, detailed pharmacy inventory, laboratory result integration, and integration with external hospital information systems; these are acknowledged as potential future extensions but are not fully specified in the current design document.

1.3 Definitions, Acronyms, and Abbreviations

Abbreviation	Term	Definition
HMS	Hospital Management System	The overall software system that manages hospital operations
SRS	Software Requirement Specifications	Formal document that describes in detail what a software system must do
JSON	JavaScript Object Notation	A lightweight, text-based data format used to represent structured data as key–value pairs and arrays
TLS	Transport Layer Security	A cryptographic protocol that provides secure communication over a computer network
HTTPS	HyperText Transfer Protocol Secure	Encrypt requests and responses between a web or mobile client and a server for protecting data
-	Carevo	Name of the Hospital Management System being designed and implemented in this project.
API	Application Programming Interface	Contract that defines how the mobile app calls backend services
OTP	One-Time Password	Password to check user identity
ID	Identification	Code specified for each user
FURPS	function, usability, reliability, performance, security	Non-functional requirements
SDK	Software Development Kit	Set of libraries and tools used to build the mobile app.
UI/UX	User Interface / User Experience	Visual layout of screens and the

		overall interaction flow in the mobile app
FCM	Firebase Cloud Messaging	Service used to send push notifications

1.4 References

Github link: <https://github.com/Rawan10101/Carevo>

System Demo: <https://youtu.be/mf28eo53y90>

Class diagram:

https://drive.google.com/file/d/1a0Nz6vGWAMX5R-RwXkGsQH0J08_pLy4/view

SRS:

https://docs.google.com/document/d/1f1hmFtX_-Dvp_DHqrPH0mdQ6GPfLzlUduLcgVieGmNs/edit?tab=t.0

Task Board Link :

<https://aucegypt-team-tk4cf28k.atlassian.net/jira/software/projects/HMS/boards/1/backlog>

1.5 Overview

This document describes the Carevo Hospital Management System (HMS) from multiple technical perspectives, guiding developers and stakeholders through how the system is structured and how it behaves. It first presents the overall system context, objectives, and scope, followed by the high-level software architecture, including the main subsystems, their responsibilities, and the client–server interaction between the mobile app and the Firebase backend.

Subsequent chapters detail the internal design of major modules such as user management, appointments, messaging, and staff scheduling, supported by class diagrams, a data dictionary for key entities (i.e. User, Patient, Appointment, Clinic, MedicalRecord, etc...), and the underlying Firestore data structures.

The document then specifies the system’s external interfaces, including user interfaces (mobile screens and flows), software interfaces (APIs between frontend and backend), hardware assumptions (phones and tablets), and communication interfaces (network protocols and security mechanisms). It also covers cross-cutting concerns such as security and safety (authentication, authorization, data privacy, audit logging), performance and scalability expectations, and availability and failure-recovery strategies based on Firebase capabilities.

Finally, the specification connects these design decisions back to the original requirements and user stories from the earlier milestones, ensuring traceability from requirements engineering through architecture and detailed design.

2. System Architecture

2.1 Design Goals & Constraints

The design of the Carevo Hospital Management System (HMS) is driven by several key goals and practical constraints. The primary design goals are **modularity and maintainability**, achieved by *separating concerns into distinct classes and subsystems for users, appointments, messaging, payments, medical records, and staff scheduling* so that each module can be evolved independently without impacting the entire system. Another major goal is responsiveness and **usability on mobile devices**, ensuring that *critical operations* such as login, profile loading, and appointment booking complete within a few seconds under normal conditions. The architecture also aims for **scalability and real-time consistency**, so that *updates to shared data* (for example, appointments or shifts) are reflected quickly for all affected users.

The main constraints come from deliberate technology choices and the project context. On the frontend, the team is constrained to a React Native + Expo mobile application, targeting standard Android and iOS phones and tablets, which influences UI patterns and available device capabilities. On the backend, the system is constrained to use Firebase Authentication and Cloud Firestore as managed services, which shape how data is modeled, how security is expressed by Firestore security rules, and how real-time updates are delivered. Additional constraints include relying on commodity mobile hardware (no special hospital devices), working within academic time and team size limits, and avoiding complex external integrations in this iteration (such as insurance and/or lab systems,), which are deferred to future versions of the architecture.

2.2 Architectural Style

We plan to use a **client-server architecture** for our system. We chose this model because our system supports multiple users: patients, doctors, nurses, receptionists, and administrators who must access and update shared hospital data in real time. For example, when a patient books or cancels an appointment, the change should immediately appear in the doctor's and receptionist's schedules, which is made possible through a centralized server. The same applies when administrators assign nurse shifts or when patients update medical information. This architecture also handles variable load efficiently since servers can be replicated as the number of users grows. While it introduces potential challenges such as network dependence or a single point of failure, it provides the best fit for ensuring secure, synchronized, and scalable communication between all hospital users.

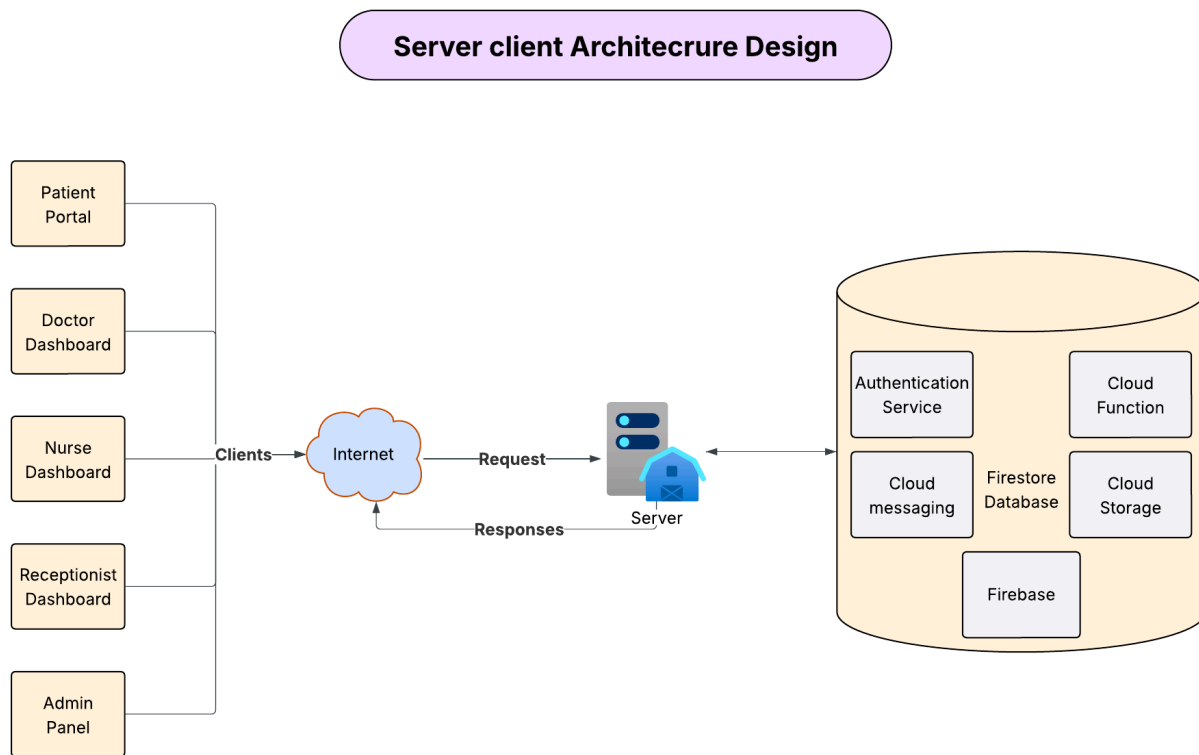


Fig 1. Server-Client Architecture Design

2.3 Module Decomposition

The major system has been broken down into modules, including authentication system, booking appointments, billing, communication & support, administrative system, shift scheduling, and medical record system. All these separate modules are centrally managed by a unified Database System and can be monitored and accessed through a central Administration Portal. This structure ensures data consistency and allows for controlled access across different user roles.

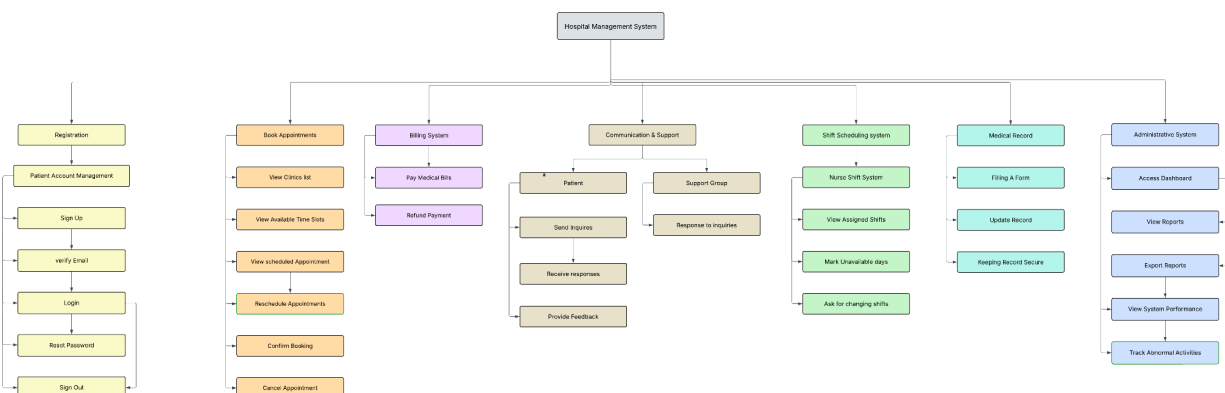


Fig 2. Overview of system modules and their relations

Furthermore, each module is broken down into tasks and components. That breakdown allows for parallel development, easier debugging, and the ability to update or scale individual system components without affecting others.

2.4 Interaction Between Modules

2.4.1 Authentication module

Coordinates all user access flows between the mobile client and Firebase Authentication. It routes sign-up, login, reset-password, and sign-out requests to internal components such as validation and token handling, stores or retrieves user credentials and profile data from the user database, and uses the email/verification service to send activation and reset links before returning success or error responses to the client.

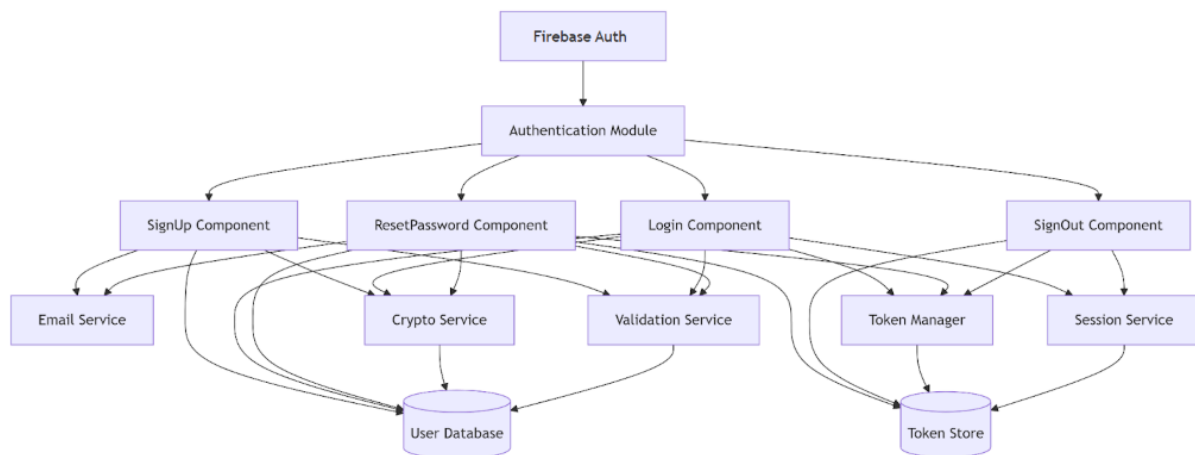


Fig 3. Overview of authentication module

2.4.2 Booking appointment module

Mediates between the patient app, the database, the scheduling data, and the notification service to complete the full booking workflow. The patient client first requests clinics, doctors, and available time slots; the module fetches this information from Firestore, then, when the user confirms a choice, it creates the appointment record, reserves the selected slot in the schedule data, and finally calls the notification service to send a confirmation message back to the patient.

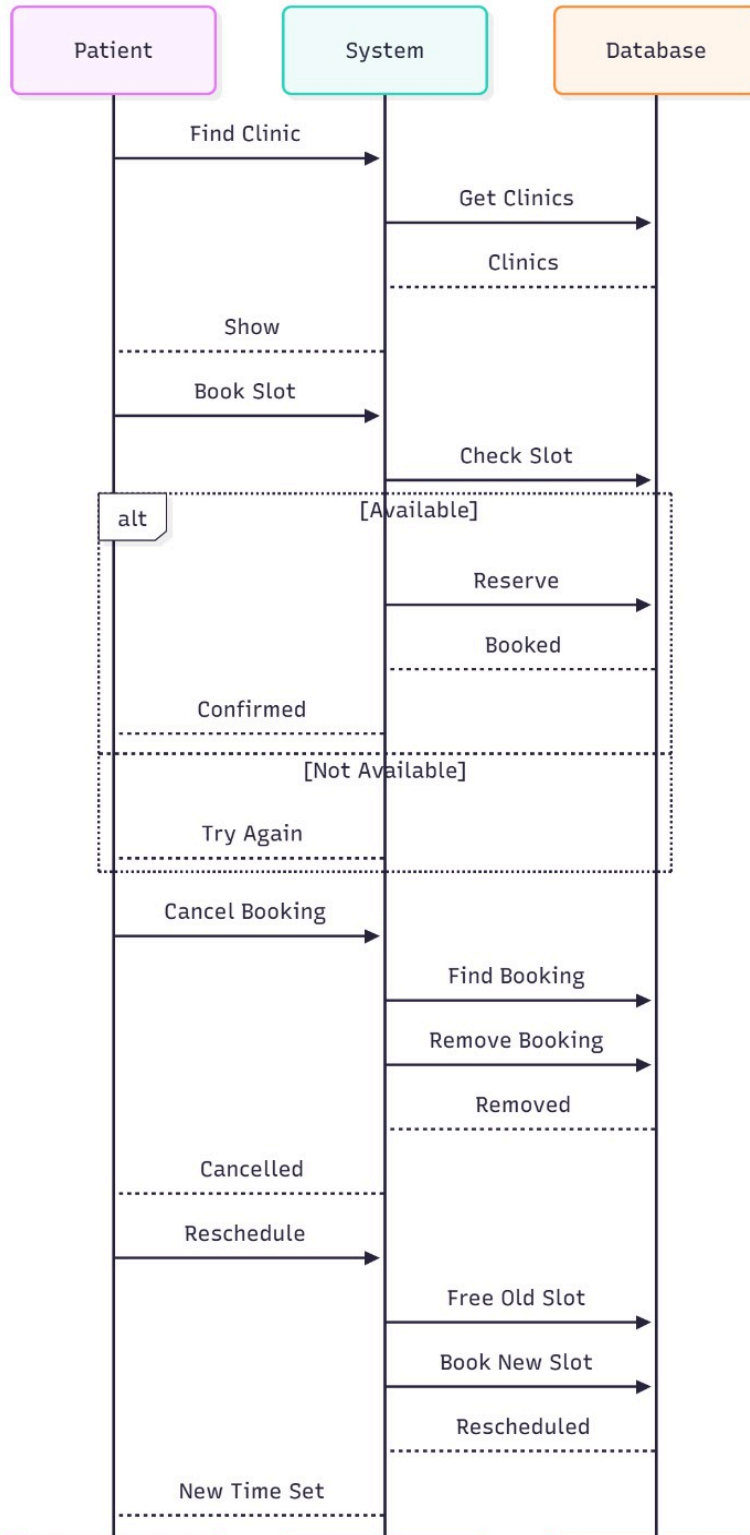


Fig 4. Sequence diagram of booking appointment.

2.4.3 Billing system module

The Billing module handles all payment-related interactions by linking the client client, the bills stored in the database, and the external payment gateway. When the client requests bills, the module queries Firestore and returns the list of unpaid items; upon a payment request, it marks the bill as in process, forwards the transaction details to the payment provider, and, if successful, updates the bill status to “Paid” in the database and returns a receipt to the user.

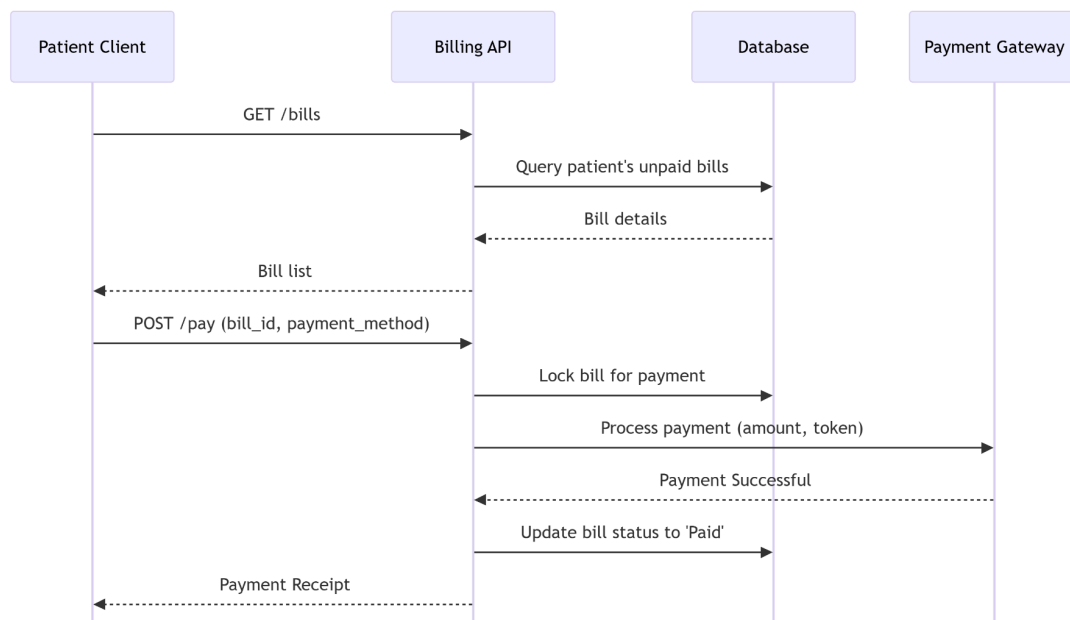


Fig 5. Sequence diagram of billing system

2.4.4 Communication and support module

Connects patients and support staff through the messaging workflow. The patient client sends a new inquiry to the module, which stores the message with metadata in the database and flags it for support; support-staff clients then retrieve messages from the same store, submit replies through the module, and those replies are saved and surfaced back to the patient, optionally triggering notifications that a response is available.

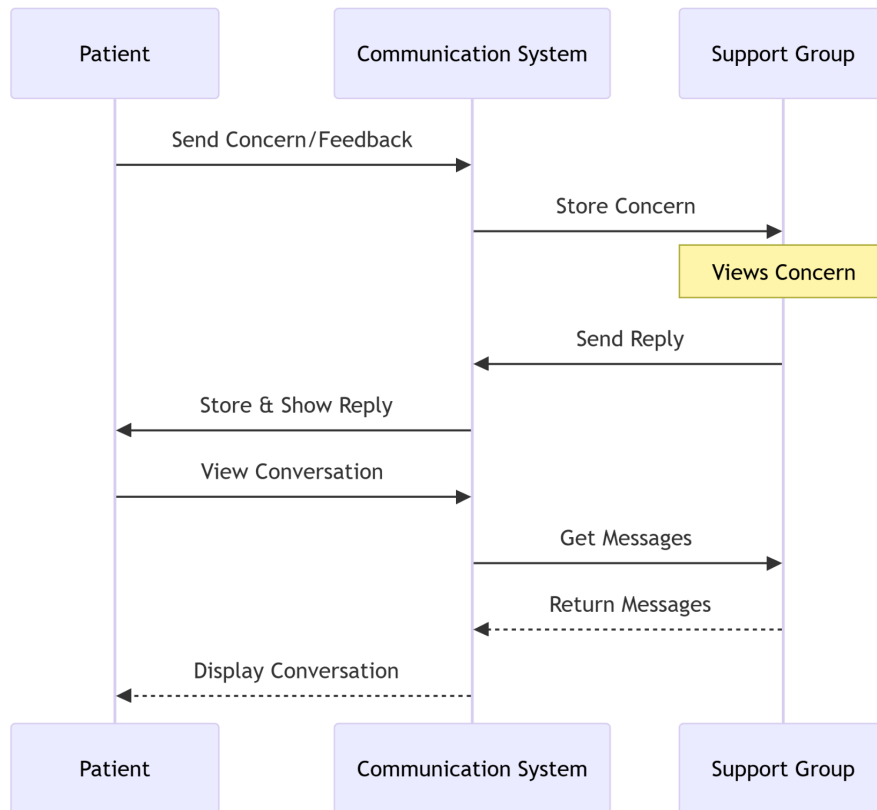


Fig 6. Sequence diagram for Communication and support

2.4.5 Shift scheduling system module

Manages nurse and doctor scheduling by mediating between admin or scheduler clients, the central schedule data, and notification services. When an admin creates or updates a shift, the module checks the database for conflicts, writes the new or updated shift record if no conflict exists, and notifies the affected staff member so their personal schedule view reflects the latest assignments.

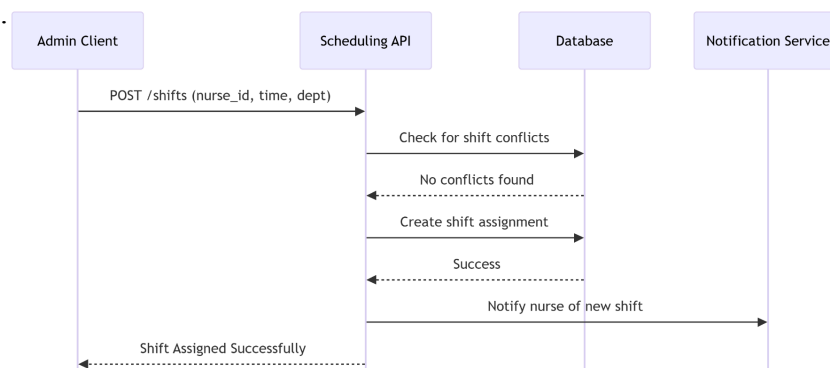


Fig 7. Sequence diagram for shift scheduling system

2.4.6 Medical record module

Coordinates access to patient records among clinician clients, security checks, and the medical-records store. For create and update operations, it validates the user's permissions, encrypts or secures the data, and writes records to Firestore, returning success messages to the UI; for viewing history, it verifies access rights, retrieves the appropriate encrypted records, filters them according to role, and returns only authorized data to the requesting doctor or nurse.

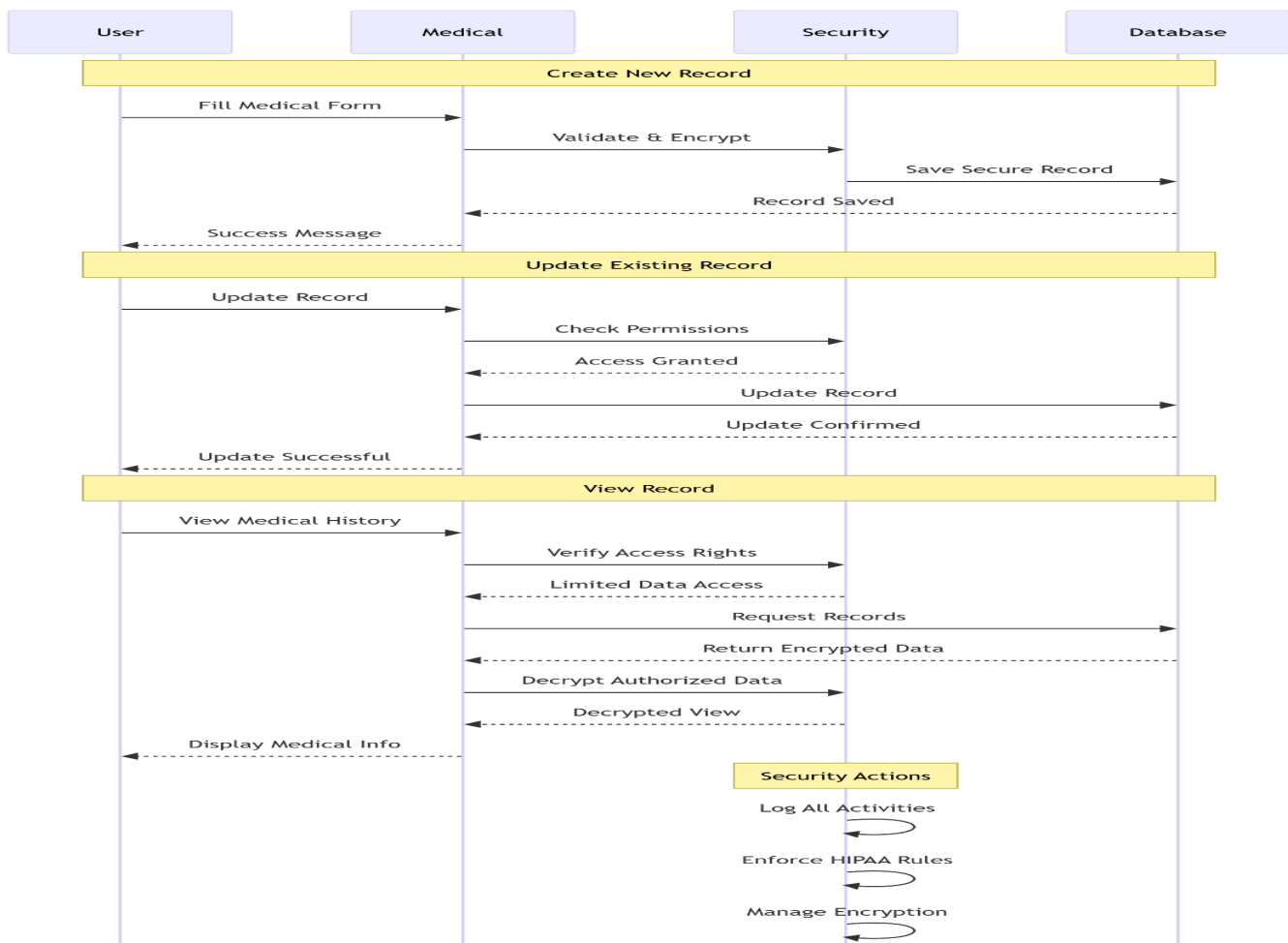


Fig 8. Sequence diagram of medical record

2.4.7 Administration System Module

Provides admin users with a consolidated view of system information and basic reporting over the shared database. It pulls aggregated metrics and historical data from multiple collections to display dashboards, generates simple reports on usage or activity when requested, and monitors system health indicators so administrators can detect issues and take action without directly querying raw patient data.

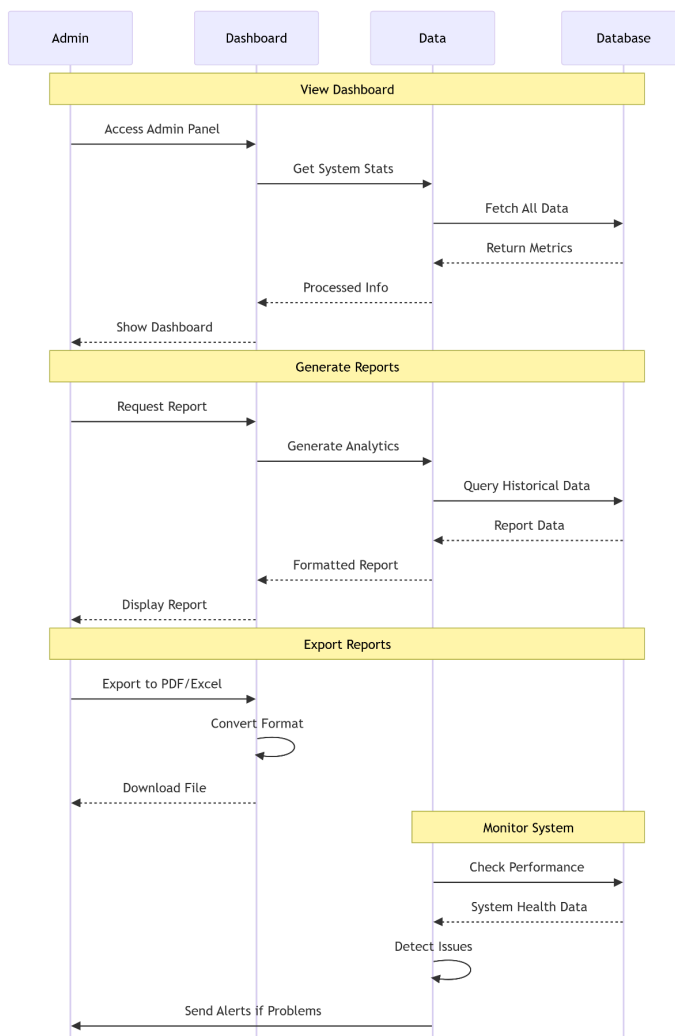


Fig 9. Sequence diagram for Administration System

3.1 Data Model / ER Diagram

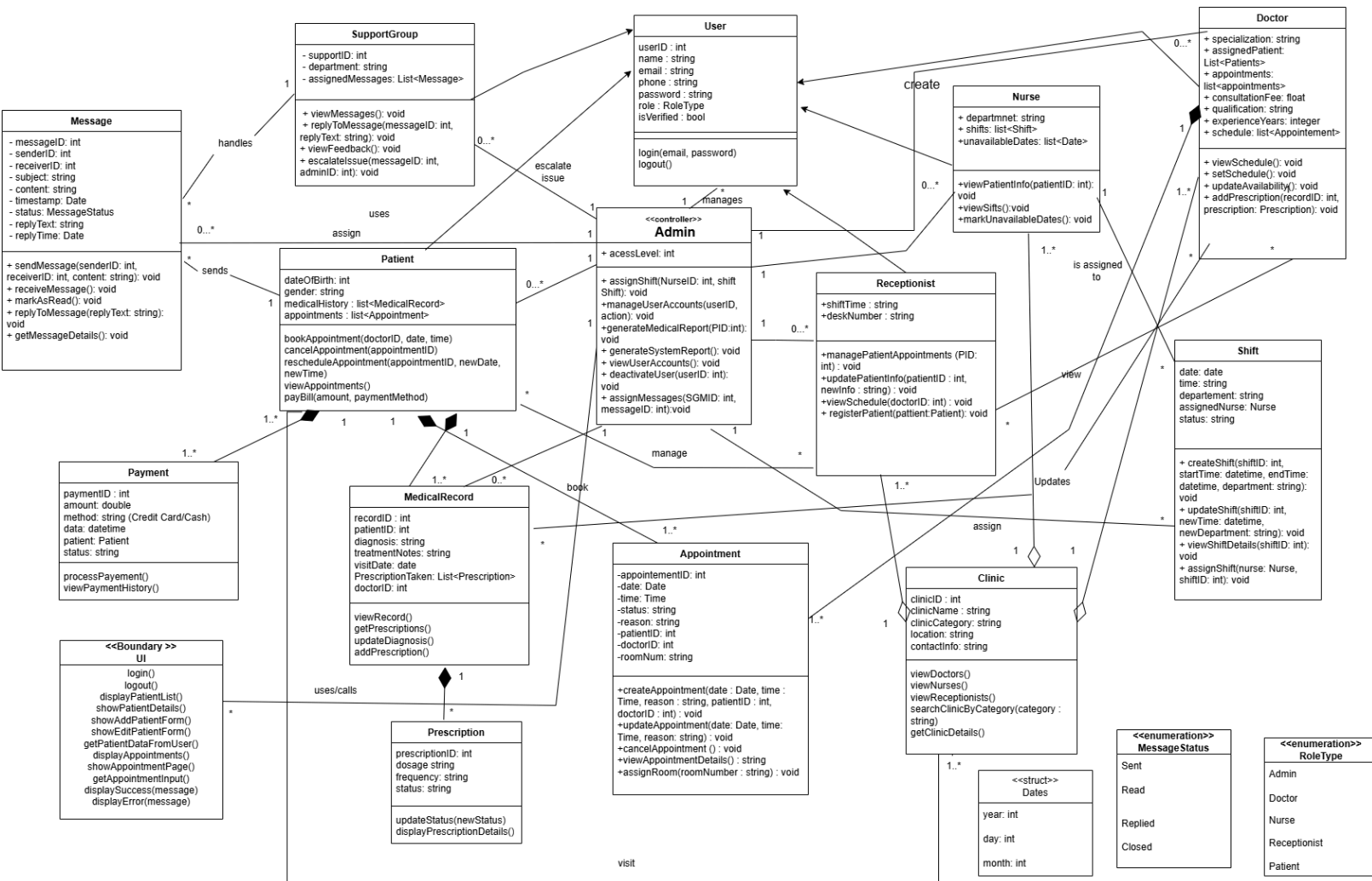


Fig 10. Class diagram for HMS

3.2 Data Dictionary

In the Carevo HMS, there are many classes that are created, each of them includes attributes that define their identities. The classes in the system are:

3.2.1 User

Role: Represents the general user that will use the system without specifying its role as each user should have these identities for **Attributes**:

- (type int) userID→ Unique identifier for each user
- (type string) name , phone “positive with no more than 11 numbers”, password “length ≥ 6 characters”
- (type RoleType) role
- (type bool) checking verification → Indicates whether the user’s account has been verified.

Operations:

- (Functions) logging-in/out of system → Authentication system

Relationships:

- Classes derived from it: supportGroup, patient, nurse, receptionist, doctor
- Association: Admin as a controller (1 to many)

3.2.2 Admin

Role: Represents a Controller who manages the system, users, clinics, and shifts, having the highest access level.

Attributes:

- (type string) accessLevel→ Describes the admin’s privilege level

Operations:

- (Void Functions) manageAccountCreation(), deactivateUser(), generateMedicalReport(), manageUserAccount(), assignMessages()

Relationships

- Association: Manages Doctor, Nurse, Receptionist Shift, medical record, patient, support group, User, prescription (1 to many)

3.2.3 SupportGroup

Role: Provides support and handles messages.

Attributes:

- (type List<Message>) list-Message → Unique identifier of the support group member.
- (type int) supportID → Department or area the support member belongs to
- (type string) department → Collection of messages currently assigned to this support member.

Operations:

- (void functions) viewMessages(), viewFeedback, escalateIssue(), replyToMessage()

Relationships

- Association: Handles Message (1 to many)
- Association: with Admin to escalate issue (many to 1)
- Inheritance: from user

3.2.4 Patient

Role: The entity receiving medical services.

Attributes:

- (type string) gender
- (type List<MedicalRecord>) history → All medical records associated with this patient.
- (type int) dateOfBirth
- (type List<appointments>) appointments → viewAppointments() – Views upcoming and past appointments.

Operations

- (void Functions) bookAppointment(), cancelAppointment(), rescheduleAppointment(), viewAppointment(), payBill(): (amount, paymentMethod)

Relationships

- Association: Sends Message (1 to many: \$1..*\$)
- Association: Assigns SupportGroup (1 to many)
- Association: with Admin (many to 1)
- Association: Receptionist (many to many)
- Composition: by MedicalRecord, appointments (1 to many), payment (many to many)
- Inheritance: from user

3.2.5 Doctor

Role: Provides medical diagnosis and treatment and handles patient appointments..

Attributes:

- (type string) specialization → The medical specialty of the doctor
- (type List<Patient>) assignedPatients → Patients currently under this doctor's care.
- (type List<Appointment>) appointments → All appointments scheduled with this doctor.
- (type float) consultationFee → The fee charged per consultation.
- (type int) experienceYears → Number of years the doctor has been practicing.
- (type List<Appointment>) schedule → The doctor's scheduled appointments.

Operations:

- (void functions) viewSchedule(), setSchedule(), updateAvailability(), addPrescription(recordID, prescription)

Relationships

- Composition: with Appointment (1 to many)
- Association: with patients (many to many) can visit many doctors.
- Association: with Admin (many to 1)
- Association: with Receptionist (many to many)
- Aggregation: with Clinic (many to 1)
- Inheritance: from User

3.2.6 Nurse

Role: Assists doctors, manages shifts, and handles patient-related tasks such as viewing schedules.

Attributes:

- (type string) department → Department the nurse belongs to

- (type List<Shift>) shifts → Shifts assigned to the nurse.
- (type List<Date>) unavailableDates → Dates on which the nurse is unavailable.

Operations:

- (void functions) viewPatientInfo(patientID), viewShifts(), markUnavailableDates()

Relationships

- Association: with Shift (1 to many) – a nurse can have many shifts.
- Association: with Admin (many to 1)
- Aggregation: with Clinic (many to 1)
- Inheritance: from User

3.2.7 Receptionist

Role: Manages patient appointments and acts as the first point of contact for scheduling.

Attributes:

- (type string) shiftTime → The time period the receptionist works in (e.g., morning, evening).
- (type string) deskNumber → The desk number or location of the receptionist.

Operations:

- (void functions) managePatientAppointments(PID, info), updatePatientInfo(patientID, newInfo), viewSchedule(doctorID), registerPatient(patient)

Relationships

- Association: with Patient, Doctor (many to many)
- Association: with Appointment (1 to many)
- Association: with Admin (many to 1) – receptionists are managed by admin.
- Aggregation: with Clinic (many to 1)
- Inheritance: from User

3.2.8 Clinic

Role: Represents a medical clinic or department where doctors provide services and appointments are booked.

Attributes:

- (type int) clinicID → Unique identifier of the clinic.
- (type string) clinicName → Name of the clinic.
- (type string) clinicCategory → Category or specialty of the clinic (e.g., Cardiology).
- (type string) location → Physical location or floor/room.
- (type string) contactInfo → Contact details for the clinic.

Operations:

- (void functions) viewDoctors(), viewNurses(), viewReceptionists(), searchClinicByCategory(category), getClinicDetails()

Relationships

- Aggregation: from Doctor (1 to many) – a clinic can have many doctors.
- Association: with Nurse (1 to many) – nurses can be assigned to specific clinics.
- Association: with Receptionist, nurse, doctor(1 to many)
- Association: with patient (many to many) – appointments take place in a clinic.

3.2.9 Appointment

Role: Represents a scheduled visit between a patient and a doctor in a clinic.

Attributes:

- (type int) appointmentID → Unique identifier of the appointment.
- (type Date) date → The date of the appointment.
- (type Time) time → The time of the appointment.
- (type string) status → Appointment status (e.g., booked, cancelled, completed).
- (type string) reason → Reason for the visit.
- (type int) patientID → Identifier of the patient.
- (type int) doctorID → Identifier of the doctor.
- (type string) roomNum → Room number where the appointment will occur.

Operations:

- (void functions) createAppointment(date, time, reason, patientID, doctorID), updateAppointment(date, time, reason), cancelAppointment(), viewAppointmentDetails(), assignRoom(roomNumber)

Relationships

- Composition: with a Patient, doctor(many to 1) – many appointments can belong to one patient & doctor.

3.2.9 Payment

Role: Represents payment transactions for patient bills.

Attributes:

- (type int) paymentID → Unique identifier of the payment.
- (type double) amount → Amount paid.
- (type string) method → Payment method
- (type DateTime) date → Date and time of payment.
- (type Patient) patient → Reference to the paying patient.
- (type string) status → Payment status (e.g., pending, completed).

Operations:

- (void functions) processPayment(), viewPaymentHistory()

Relationships

- Composition: with Patient (many to 1) – a patient can have many payments.

3.2.10 Shift

Role: Represents a work shift for a nurse.

Attributes:

- (type Date) date → Date of the shift.
- (type Time) startTime → Start time of the shift.
- (type Time) endTime → End time of the shift.
- (type string) department → Department of the shift.
- (type Nurse) assignedNurse → The nurse assigned to this shift.
- (type string) status → Status of the shift (e.g., assigned, unassigned).

Operations:

- (void functions) createShift(), updateShift(), viewShiftDetails(), assignShift(nurse, shiftID)

Relationships

- Association: with Nurse (many to 1) – many shifts can be assigned to one nurse over time.
- Association: with Admin (many to 1) – shifts are created and managed by admin.

3.2.11 Message

Role: Represents communication between patients and the support group (or other staff).

Attributes:

- (type int) messageID → Unique identifier of the message.
- (type int) senderID → Identifier of the sender (patient, support, admin, etc.).
- (type int) receiverID → Identifier of the receiver.
- (type string) subject → Subject line of the message.
- (type string) content → Message body.
- (type Date) timestamp → When the message was sent.
- (type MessageStatus) status → Message status (Sent, Read, Replied, Closed).
- (type Date) replyTime → Time at which a reply was sent (if any).

Operations:

- (void functions) sendMessage(), receiveMessage(), markAsRead(), replyToMessage(), getMessageDetails()

Relationships

- Association: with Patient (many to 1) – patients can send many messages.
- Association: with SupportGroup (many to 1) – support members handle many messages.

3.2.12 MedicalRecord

Role: Stores clinical information about a patient's diagnosis, treatment, and visit history.

Attributes:

- (type int) recordID → Unique identifier of the medical record.
- (type int) patientID → Identifier of the patient.
- (type string) diagnosis → Diagnosis text.
- (type string) notes → Additional notes about the visit.
- (type Date) visitDate → Date of the visit.
- (type List<Prescription>) prescriptionTaken → List of prescriptions linked to this record.
- (type int) doctorID → Identifier of the doctor who created the record.

Operations:

- (void functions) viewRecord(), addDiagnosis(), updateDiagnosis(), addPrescription()

Relationships

- Composition: by Patient (1 patient to many medical records).
- Association: with Doctor (many to 1) – records are created by doctors.
- Association: with Admin (many to 1)
- Composition: with Prescription (1 to many)

3.2.13 Prescription

Role: Represents medication prescribed to a patient as part of a treatment plan.

Attributes:

- (type int) prescriptionID → Unique identifier of the prescription.
- (type string) dosage → Dosage instructions (e.g., “2 tablets daily”).
- (type string) drugName → Name of the prescribed drug.
- (type string) status → Status of the prescription (e.g., active, completed).

Operations:

- (void functions) updateStatus(newStatus), displayPrescriptionDetails()

Relationships

- Composition: with MedicalRecord (many to 1) – multiple prescriptions can belong to one medical record.

3.2.14 UI

Role: Represents the user interface layer that interacts directly with system users. Also, it is responsible for displaying the screens.

Operations:

- Operations (UI actions):
- loginUI() – Displays login form and forwards credentials to User.login.
- logoutUI() – Triggers logout and returns user to login screen.
- showPatientList() – Requests and displays patient list to staff (for receptionist/admin views).

- showPatientDetails(PID: int) – Displays detailed information for a selected patient.
- showAddPatientForm() – Presents patient registration form and passes submitted data to Admin/Receptionist.
- showAppointmentForm() – Displays appointment booking form and forwards data to Patient.bookAppointment.
- displaySuccess(message: string) – Renders confirmation messages to the user.
- displayError(message: string) – Renders validation or system error messages.

Relationships

- Association: with Admin (many to 1)

3.3 Persistence / Storage Strategy

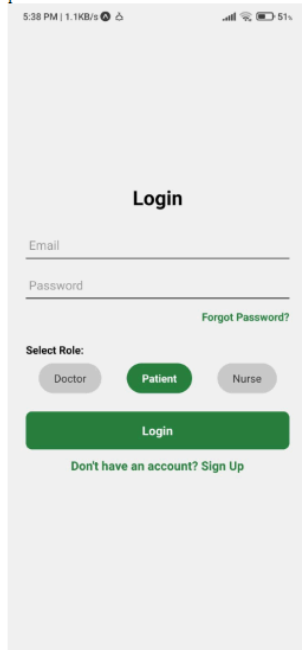
The Carevo Hospital Management System implements a comprehensive persistence and storage strategy utilizing Google Firebase as the core backend infrastructure. For data storage, Cloud Firestore serves as the primary NoSQL document database, providing real-time synchronization and flexible data modeling for patient records, appointments, and medical data. User authentication and security are managed through Firebase Authentication, which handles user registration, login sessions, and access control. Server-side business logic is implemented using Firebase Cloud Functions for handling complex operations like payment processing and notification services. The frontend is developed using React Native with Expo framework, enabling cross-platform mobile application development. The development environment utilizes Visual Studio Code as the primary IDE, with Expo CLI and Expo Go facilitating application testing and deployment. Version control is managed through Git and GitHub for source code management and team collaboration. The testing strategy employs both simulators/emulators during development and real-device testing using Expo Go for production validation, ensuring application reliability across different mobile platforms.

4. Interface Design

4.1 User Interfaces

1. **Login Screen**

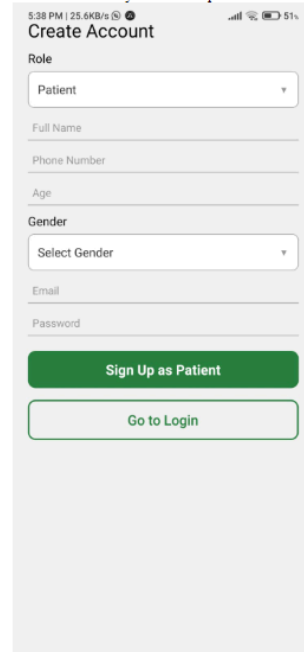
Provides a secure interface where users enter their email and password to access the Carevo system and navigate to their personalized dashboard.



The Login Screen features a light gray background. At the top, the status bar shows the time as 5:38 PM, signal strength, and a 51% battery level. The title "Login" is centered in a bold, black font. Below the title, there are two input fields: "Email" and "Password", each with a green underline. A "Forgot Password?" link is positioned to the right of the Password field. Below the input fields, there is a "Select Role:" section with three buttons: "Doctor", "Patient" (which is highlighted in green), and "Nurse". At the bottom, there is a large green "Login" button and a link that says "Don't have an account? Sign Up".

2. **Patient Sign-Up Screen**

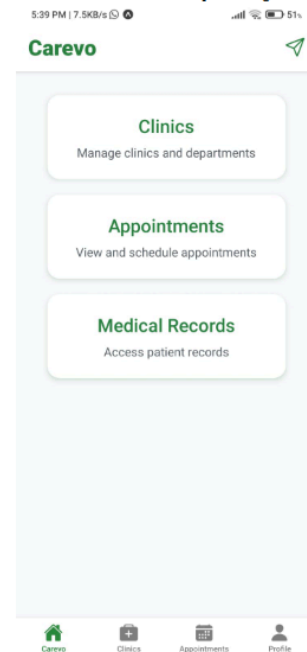
Allows new patients to create an account by selecting the patient role and entering their personal and contact information so they can use hospital online services



The Patient Sign-Up Screen has a light gray background. The status bar at the top shows 5:38 PM, signal strength, and a 51% battery level. The title "Create Account" is centered in a bold, black font. Below the title, there is a "Role" dropdown menu with "Patient" selected. Below this, there are four input fields: "Full Name", "Phone Number", "Age", and "Gender" (which has a "Select Gender" dropdown). Below the input fields, there are two more input fields for "Email" and "Password". At the bottom, there are two buttons: a green "Sign Up as Patient" button and a white "Go to Login" button with a green border.

3. **Patient Home Dashboard**

Displays the main entry point for patients, summarizing key modules such as clinics, upcoming appointments, and access to medical records for quick navigation.



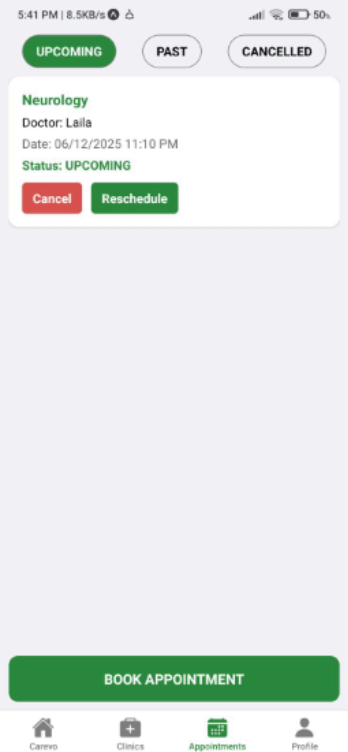
The Patient Home Dashboard features a light blue background. The status bar at the top shows 5:39 PM, signal strength, and a 51% battery level. The title "Carevo" is at the top left in a bold, green font, with a green arrow icon to its right. Below the title, there are three large, rounded rectangular buttons: "Clinics" (with the subtext "Manage clinics and departments"), "Appointments" (with the subtext "View and schedule appointments"), and "Medical Records" (with the subtext "Access patient records"). At the bottom, there is a navigation bar with four icons: a green house icon for "Carevo", a green building icon for "Clinics", a green calendar icon for "Appointments", and a green person icon for "Profile".

4. **Clinics List Screen**

Fig 11. UI for login, sign_up, and patient dashboard

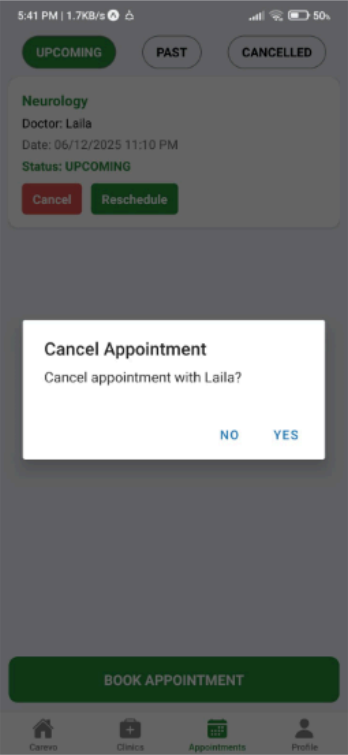
7. Cancel / Reschedule Appointments Screen

Displays all appointments available that were booked by the patient or staff, allowing patients to reschedule / cancel this appointment if needed



8. Cancel Appointment Confirmation Dialog

Asks the patient to confirm cancellation of a selected appointment, warning that the time slot will be released so others can book it.



9. Cancelled Appointments Screen

Displays all appointments that were cancelled by the patient or staff, allowing patients to review their history of cancelled visits separately from active ones.

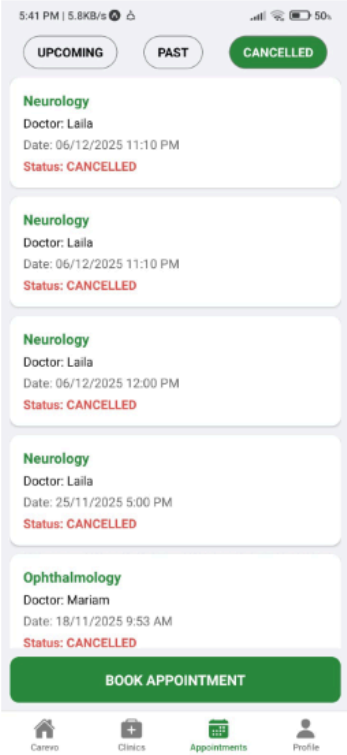
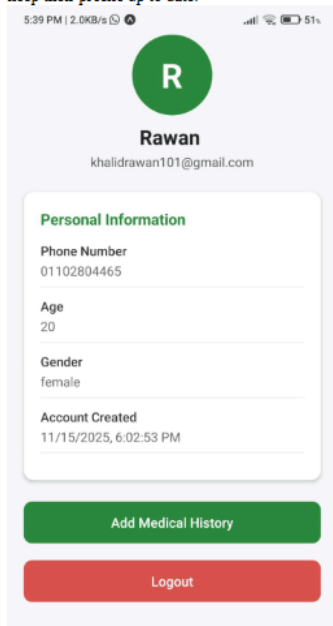


Fig 12. Booking appointments, canceling appointments, and confirmation messages

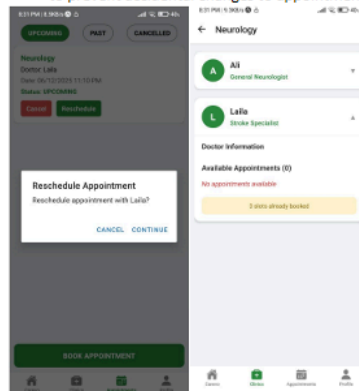
10. Profile Screen (Patient)

Shows the patient's personal information such as name, contact details, and basic demographics, enabling them to review and keep their profile up to date.



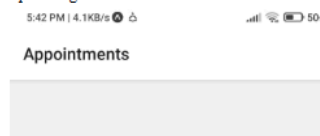
11. Reschedule Confirmation Dialog

Prompts the patient to confirm before applying a reschedule request, summarizing the new date and time to prevent accidental changes to appointments.



13. Doctor Appointments Screen

Shows all appointments scheduled with the logged-in doctor in chronological order, helping them prepare for upcoming visits.



12. Doctor Sign-Up Screen

Allows doctors to register in the system with their professional details so they can later manage schedules and view assigned appointments.

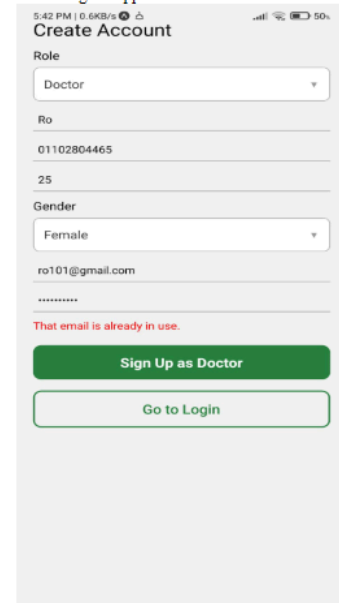


Fig 13. Profile page, booked appointments screen in patients account, and doctors' signup screen

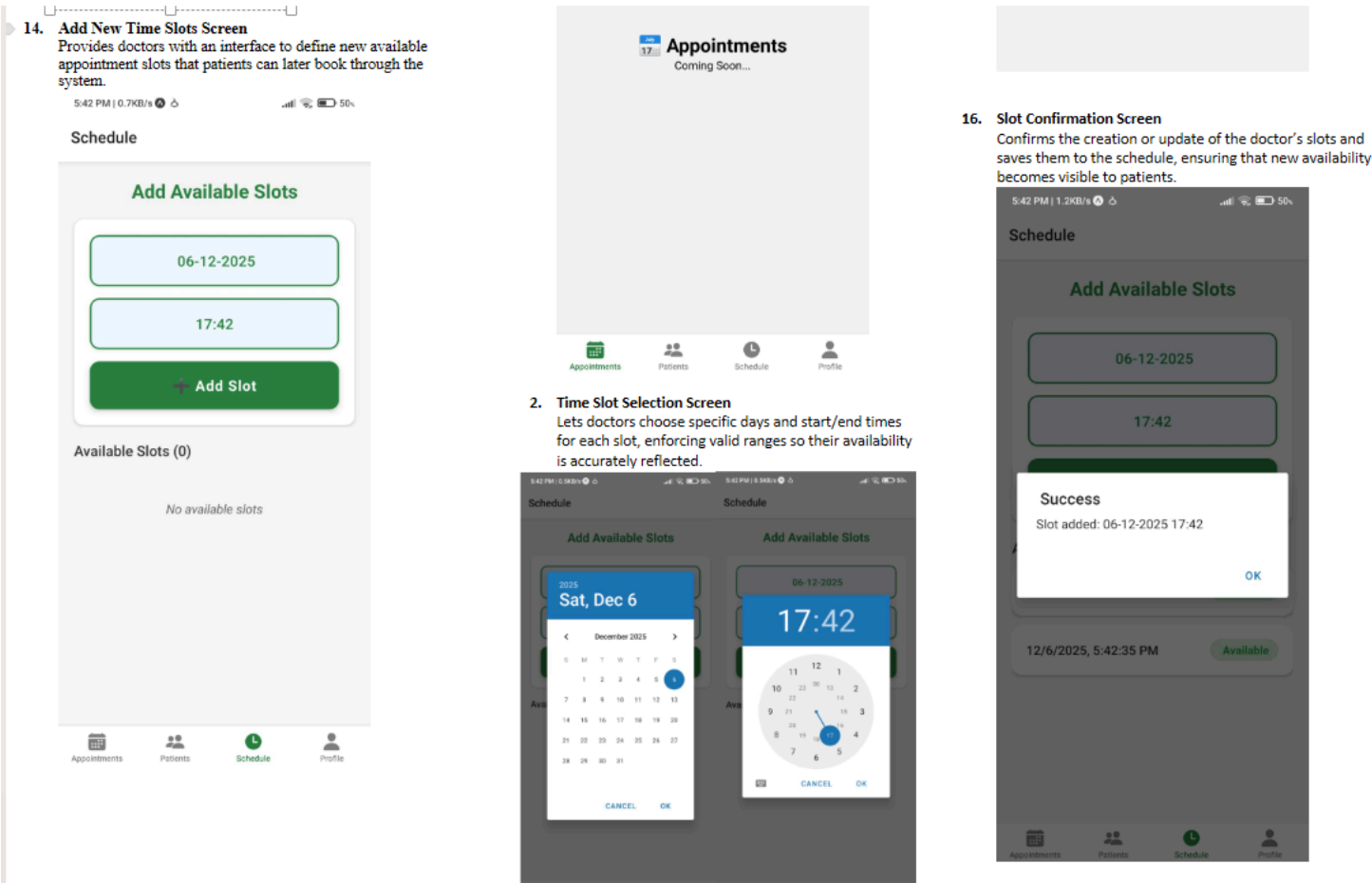


Fig 14. UI for adding new slots by doctors, the doctor’s dashboard, and the success message after adding a slot.

4.2 API / Software Interfaces

Carevo system employs a client-server architecture in which React Native frontend connects directly with Firebase backend service. The Firebase backend is the Firebase Authentication and Firestore Database. All logic executes client-side using Firebase SDKs. Moreover, the system uses a modular component-based design with clear separation between authentication, data management, and user interface layers. The data format was by using JSON for all request bodies and responses. In addition, Firebase handles the Authentication cases using Firebase Authentication since it controls data access by its security rules in Firestore. In this section, the endpoints, parameters, request and response formats, and error handling required to support the frontend-backend integration for the hospital management system, reflecting the current implementation and immediate roadmap features.

4.2.1 Authentication and User Management Interfaces

Different interfaces were made in this section. API handles sign up, login, logout, and forget password functionalities for the user.

- Sign-Up API: creates new patients by accepting user details and credentials, then stores the profile in Firestore. Input validation ensures fields such as email, password (min length 6), age, gender, and phone number are properly given. The backend uses Firebase Auth for credential management and Firestore for profile data storage. Appropriate error messages are returned for duplicate emails or weak passwords.
- Login API: authenticates users based on email and password, establishing a session on success and returning descriptive error messages on failure.
- Logout API: terminates the user session and navigates the user back to the login interface securely.
- Forget Password API: enables the user to enter their email address to receive notification about the OTP to check their account reliability, ensuring security, and making them create a new password.

4.2.2 Patient Profile

The patient profile API retrieves user details such as name, age, gender, phone, and registration date, enabling the frontend to display personalized information in the profile screen. Profiles can be fetched using the authenticated user's unique ID. All profile data transfers are encrypted and validated to maintain privacy and integrity.

4.2.3 Clinics and Doctors Listing

The system provides APIs to list clinics and associated doctors for appointment booking.

- Clinics: can be queried optionally by category, returning details such as clinic name, department, and location to populate clinic selection screens.
- Doctors: are listed by the selected clinic, with details about their specialty and availability summary. This hierarchical querying supports the user journey from clinic selection to doctor and time slot selection.

4.2.4 Appointment Management

Appointment APIs cover creation, retrieval, and cancellation of patient appointments.

- Available Slots API exposes doctors' daily time slots, filtered by date and doctor ID, showing only free slots for booking.
- Booking API enables patients to reserve an available slot, resulting in a new appointment record and marking the slot as unavailable to prevent conflicts.
- Retrieval API fetches all appointments for a patient, segmented by status (upcoming, past, cancelled), providing a complete overview of their visits.
- Cancellation API allows patients to cancel appointments, updating status and freeing time slots in doctor schedules accordingly. These operations ensure data consistency and timely UI updates.
- Rescheduling API enables patients to modify existing upcoming appointments by selecting a new available time slot. The system cancels the original appointment, creates a new one in the selected slot, updates the doctor's schedule for both old and new times, and sends confirmation notifications to both patient and doctor.
- View Past appointment API displays the past appointments handled in a separate section with full details.

Moreover, APIs for doctors and receptionists to view relevant appointment schedules support administrative workflows.

4.2.5 Error Handling

The system returns structured error responses with clear, user-friendly messages for authentication errors and general backend issues. Firebase Auth specific errors like invalid email or weak password are mapped to descriptive prompts. Network or permission errors are logged and displayed with generic messages, ensuring robust front-end user experience during asynchronous operations.

4.3 Hardware Interfaces (if applicable)

Since the Carevo Hospital Management System is built using Reactive Native with Expo Go, the user can use different hardware platforms to access the application, like smartphones and/or

tablets. Accessing the application is easy since the user will not need any more specialized medical hardware to access it.

The supported hardware platforms can work on:

- Smartphones: (iPhone 12 and newer), Android (API level 26+). Minimum screen size 5.5 inches.
- Tablets: iOS iPad (10th gen+), Android tablets (8 inches+). Optimized for larger screens via responsive design.
- For the developer: they can access it through Expo Go app for testing.
- There is no interaction with printers, scanners, medical sensors, or any specialized medical hardware.
- All the operations are done via the Firebase backend.

The application leverages standard mobile hardware interfaces by having:

- Touchscreen: since it provides support for navigation, input data, and doing different selection processes (scheduling appointment, rescheduling, or cancelling, and navigating through different screens).
- Network Interface: WiFi/4G/5G for Firebase cloud communication, since the application needs internet to open and work.

Moreover, there are some requirements needed to be in the hardware device, such as the minimum specifications for these requirements are:

- Processor: isQuad-core 2.0GHz
- RAM: 3GB min
- Storage: 100MB free
- Screen: touch screen with 5.5" 720p
- Network: WIFI/4G
- OS: iOS 14+, Android 10+

4.4 Communications / Network Interfaces

The Carevo Hospital Management System uses HTTPS (HyperText Transfer Protocol Secure) over standard internet protocols to communicate with Firebase cloud services (Authentication and Firestore). All data exchanges use JSON payloads with end-to-end TLS (Transport Layer Security) encryption.

4.4.1 Protocols and Message Formats

The format used in the system is:

- For the primary protocol: HTTPS (TLS 1.3) on port 443 for all API calls is identified.
- For the secondary protocol: WebSocket (wss://) for Firestore real-time listeners is used.
- Message format: is done through JSON document (*Content-Type: application/json*)
- Authentication: is done using Firebase JWT tokens which are automatically managed by SDK.

4.4.2 Security Mechanisms

Different layers of security are used to ensure security by putting data encryption standards. That will ensure safety and avoid any data breach that might occur. The system is secured by:

- Transport Security: TLS 1.3 encryption for all traffic
- Authentication: Firebase Auth tokens with automatic refresh
- Authorization: Firestore Security Rules (per-user document access)
- Data Protection: Firebase-managed encryption at rest

4.4.3 Bandwidth Considerations

Operation	Size	Network required
Signup/Login	Around 2KB	min 3GB
Profile	Around 1KB	No specification
Clinics/Doctors	5-20KB	3G/4G
Appointments	3-15KB	4G/WIFI
Real-time Sync	<1KB	WIFI/mobile data

4.4.4 Error Handling and Reliability

Handling the error is done making:

- Timeouts: 30s (API), 10s (real-time listeners)
- Retries: Exponential backoff for transient failures
- Offline Support: Firebase SDK caching + queued operations
- Error Codes: Standard HTTP (401, 429, 500) mapped to user-friendly messages
- Error Codes: Standard HTTP (401, 429, 500) mapped to user-friendly messages

5. Component Design & Algorithms

5.1 Component Descriptions

5.1.1 User & Authentication Component

Registration, login, logout, and role-based access are handled here for all user types (patients, doctors, nurses, receptionists, admins, support group). It encapsulates the User base class and derived classes (e.g.Patient, Doctor, Nurse, Receptionist, Admin, SupportGroup), along with Firebase Authentication calls for creating accounts and verifying credentials. Internally, it manages user attributes (ID, name, email, phone, role, verification status), performs input validation on signup/login screens, and exposes operations such as login, logout, and manageUserAccounts that other components call via the mobile UI.

5.1.2 Appointment Management Component

The Appointment Management component is responsible for the full lifecycle of appointments, including creation, rescheduling, cancellation, and retrieval for both patients and staff. It is implemented primarily through the Appointment class and its associations with Patient, Doctor, Clinic, and Shift, supported in the frontend by the clinics, doctor list, and appointment screens. Internally, it enforces rules such as preventing double-booking, updating doctor and receptionist views when a slot changes, and coordinating with the payment and notification flows; key operations include createAppointment, updateAppointment, cancelAppointment, and viewAppointments.

5.1.3 Shift Scheduling Component

This component manages work schedules for nurses and doctors, ensuring coverage and respecting unavailability constraints. It is built around the Shift and Nurse classes, with the Admin acting as controller to create, assign, and update shift records. The internal logic validates time ranges, prevents overlapping shifts for the same nurse, and updates nurses' schedule views; core operations include createShift, updateShift, assignShift, and viewShifts.

5.1.4 Messaging & Support Component

The Messaging & Support component enables patients to send inquiries and feedback and allows support staff to handle and escalate them. It uses the Message and SupportGroup classes, with relationships to Patient and Admin for escalation when needed. Internally, it stores message content, timestamps, and status, and provides operations such as sendMessage, viewMessages,

replyToMessage, and escalateIssue, which are coordinated through the UI boundary screens for messaging.

5.1.5 Medical Record & Prescription Component

This component manages patient medical history and prescriptions for clinical decision support. It centers on the MedicalRecord and Prescription classes, linked to Patient and Doctor, and is accessed through doctor- and nurse-facing views rather than the main patient UI. Its internal responsibilities include storing diagnoses, visit dates, treatments, and prescription details, as well as providing read-only access for authorized roles; operations include viewRecord, updateDiagnosis, and addPrescription, while enforcing privacy and immutability rules for historical entries.

5.1.6 Billing & Payment Component

The Billing & Payment component handles recording and processing of patient payments for hospital services. It uses the Payment class associated with Patient and Appointment, and exposes operations such as processPayment and viewPaymentHistory. Internally, it records the amount, method (credit card, cash, etc.), date, status, and links to the relevant appointment, while delegating actual payment processing to external providers in future iterations; in the current design it focuses on logging payment information and updating patient billing views.

5.2 Key Algorithms

There are different algorithms used in constructing the system.

5.2.1 Appointment Conflict Detection

When a patient books or reschedules an appointment, the system checks for time conflicts before confirming the request. The algorithm queries existing appointments for the selected doctor and time range; if any appointment overlaps with the new start/end time, the booking is rejected and an error message is shown. If no overlap exists, the appointment is created and linked to both the patient and doctor, and the corresponding time slot is marked as unavailable so future bookings cannot reuse it. This logic ensures that double-booking of the same doctor, room, or time slot does not occur.

5.2.2 Appointment Rescheduling Logic

Rescheduling is treated as an atomic operation consisting of conflict checking, cancellation of the old appointment, and creation of a new one. First, the system verifies that the requested new slot is available using the same conflict detection algorithm. If valid, it updates the old

appointment's status to "rescheduled" or "cancelled," frees its time slot, then inserts a new appointment with the new date and time while updating the doctor's and receptionist's views. This guarantees consistency between the doctor's schedule, the patient's list of appointments, and future availability.

5.2.3 Shift Assignment and Overlap Prevention

For nurse and doctor shifts, the system validates new or updated shifts to prevent overlapping assignments and invalid time ranges. When an admin assigns a shift to a nurse, the algorithm checks existing shifts for that nurse in the same date range; if any overlap is detected, the assignment is rejected and the admin is notified. It also ensures that the start time is earlier than the end time and that a nurse is not assigned to two departments at the same time. This maintains a consistent schedule and supports safe staffing levels.

5.2.4 Role-Based Access Control Evaluation

Every sensitive operation (such as viewing medical records, accessing appointment lists, or managing user accounts) passes through a role check based on the user's RoleType. When a request is made, the algorithm evaluates the user's role against allowed roles for that action. Combined with Firestore security rules, this logic ensures that each user can only access data appropriate to their role.

5.2.5 Doctor Availability Validation

When a doctor creates or edits their schedule, the system validates each new slot to ensure that the start time precedes the end time and that the slot does not overlap with existing availability or confirmed appointments. If a conflict is detected, the change is rejected and an error message is shown; otherwise, the slot is stored and becomes visible to the patient booking flow, which only offers these validated slots for new appointments.

6. Security & Safety

6.1 Authentication & Authorization

The Hospital Management System (HMS) ensures the security and safety of user data through multiple measures. Authentication and authorization are handled using Firebase Authentication, allowing only registered users to log in and access the system. Different roles, such as patients and administrative staff, have specific permissions to ensure that sensitive data is accessed only by authorized users. Only verified users are allowed to access the system. Data privacy and encryption are maintained by using HTTPS for all communications between the mobile application and Firestore, and sensitive information is stored securely in the cloud. Personal and medical data are protected from unauthorized access, and local storage on devices is minimized and encrypted when necessary. In addition, automatic token refresh and secure logout clears all credentials.

6.2 Data Privacy & Encryption

The Hospital Management System (HMS) protects personal and medical data by using secure cloud storage through Firestore. All data transmitted between the mobile application and Firestore is encrypted using HTTPS, ensuring encryption in transit. Sensitive information stored in the cloud is encrypted at rest according to Firebase security standards. Local storage on devices is minimized, and any data temporarily stored is encrypted to prevent unauthorized access. In addition, data is isolated by user ID, so there is no cross-patient access or any data mismatch.

6.3 Audit Logging & Monitoring

The logged operations are:

Operation	Logged details	Retention
Log-in/out	User ID, timestamp, IP, success/failure	90 days
Appointment Book/schedule/cancel	Patient ID, doctor ID, appointment ID, timestamp	2 years
Profile access	User ID, document read timestamp	30 days
Data modification	User ID, field changed, old/new values, timestamp	1 year

Firebase functions can trigger the cloud logging on critical events. Firestore “createdAt” /”updatedAt” timestamps can track any changes in the document.

Access to Log is given to different trusted roles, since we have:

- Admins: who are fully audited for handling it via Firebase console
- Developers: can read-only logs by using Google cloud logging
- Patients: can access their own appointment histories only
- Monitoring: is a tracker to track Firebase performance and the error time rate

6.4 Failure Modes & Recovery

The HMS is designed to handle errors, system crashes, network failures, and potential data corruption gracefully. In case of temporary network issues, the application allows offline access and synchronizes changes once connectivity is restored. Firestore provides built-in backup and redundancy to ensure minimal data loss. If a system error occurs, users receive clear error messages, and operations can be retried safely without compromising data integrity.

7. Performance, Scalability & Availability

7.1 Performance Requirements

The Carevo Hospital Management System (HMS) is designed to be responsive and efficient on standard mobile devices. Screens should load within 2 seconds, and key actions such as booking or rescheduling appointments should complete with minimal delay (i.e. ≤ 3 seconds). Data synchronization with Firestore must occur in near real-time, within 5 seconds, to ensure users always see accurate appointment and doctor availability information. The system is optimized to handle multiple concurrent users without noticeable slowdowns under normal usage conditions. Loading indicators provide feedback during async operations.

7.2 Scalability

The system is designed to scale primarily through Firestore's cloud infrastructure. As the number of users or appointments grows, Firestore can handle additional data and traffic without requiring significant changes to the application. The system supports horizontal scaling by allowing multiple devices to access the database simultaneously, ensuring smooth performance as the user base increases. That means there are no backend servers to manage. Also, the specific users' IDs ensures there is no conflict in data as the hospital network grows.

7.3 Availability & Fault Tolerance

The HMS is intended to be highly available during operational hours. Firestore provides built-in redundancy and backup mechanisms to protect against data loss. Also, Atomic transactions prevent race conditions during simultaneous booking attempts. In case of network disruptions, the application will handle temporary offline states and synchronize changes once the connection is restored. 24-hour point-in-time recovery available through Firebase backups. Basic failover strategies rely on Firestore's cloud reliability, ensuring that users can access the system and their data consistently without interruptions.

8. Design Risks and Mitigations

8.1 Third-Party Backend Dependency (Firebase)

The system depends heavily on Firebase Authentication and Cloud Firestore for all authentication, data storage, and real-time updates, which introduces vendor lock-in and risk if the service experiences outages, pricing changes, or quota limits.

To mitigate this, all backend access is encapsulated behind service abstractions so that calls to Auth and Firestore are not scattered throughout the UI logic, making it easier to replace or augment Firebase in the future. The team also monitors usage and performance from the Firebase console and documents data models and API contracts so that migration to another backend is feasible if required.

8.2 Firestore Query Performance and Data Growth

As the number of appointments, messages, and medical records grows, inefficient Firestore queries (missing indexes, large unfiltered collections) could degrade performance for features such as clinic browsing, appointment history, and search. This risk already appeared when integrating more complex appointment filtering.

The design mitigates this by modeling data according to access patterns (patient-scoped appointment collections, indexed fields for common filters) and limiting query result sizes with pagination where needed. Planned refactoring of Firestore queries and continuous monitoring of slow operations are included as explicit improvement items in later sprints.

8.3 Security Misconfiguration and Unauthorized Access

Incorrect Firestore security rules or missing role checks in the app could allow users to access or modify data outside their permissions, such as viewing other patients' records or altering schedules. This is critical given the sensitivity of medical and billing information described in the requirements.

Mitigation includes enforcing role-based access control via Firestore security rules aligned with the RoleType model, validating permissions in the application layer, and logging access to sensitive collections like medical records and payments. Regular negative testing (attempting forbidden operations) and review of audit logs are planned as part of the testing and quality activities.

8.4 Inconsistent Schedules and Appointment State

Because appointment booking, rescheduling, cancellation, and doctor/nurse schedules are maintained in related but separate records, there is a risk of inconsistent state, like a slot shown as free while already booked, or appointments not reflected correctly in doctor and receptionist views. This was highlighted in user stories for rescheduling and shift assignment.

To mitigate this, the design uses atomic operations or transactions whenever both an appointment document and a corresponding schedule slot must be updated together, and core invariants (one slot can only be in one state) are enforced in the Appointment and Shift components. Integration and system tests specifically exercise booking, rescheduling, and cancellation flows to ensure all related views remain synchronized.

8.5 Offline Usage and Synchronization Conflicts

The mobile app is expected to be used over unstable networks, and Firebase's offline caching can create conflicts if multiple users act on the same data, like two patients booking the last slot, while offline.

The mitigation strategy is to treat the backend as the source of truth and always revalidate critical actions, like booking or rescheduling, against the latest server state when connectivity is available, showing clear error messages if an offline change cannot be applied. The UI also distinguishes between online and offline states and avoids committing high-risk operations when the server cannot confirm success.

8.6 Mobile UI Complexity and Usability Issues

As more features are added (forgot password, rescheduling, viewing past appointments, messaging, payments), the mobile interface may become cluttered or confusing, leading to user errors and longer task completion times. This risk was visible when the team had to refine navigation and validation flows in later sprints.

Mitigation involves following consistent UI patterns across screens, using clear labels and confirmation dialogs for destructive actions (like cancel or reschedule), and validating input aggressively on forms such as sign-up and payment. Feedback from sprint demos and system testing is used to iteratively refine screen flows and reduce cognitive load for patients and staff.

9. Testing & Verification Strategy

9.1 Unit and Integration Testing

This foundational phase verifies the technical correctness of the application's underlying code and inter-component communication.

Unit Testing

Unit testing focuses on validating the smallest testable components or functions in **complete isolation**, independent of external services (like the database or network). This ensures the core business logic is sound before integration.

- **Objective:** To verify the logic of application features.
- **Application Focus:** Rigorously testing core validation methods and reusable utility functions to ensure they conform precisely to business rules. This confirms the mathematical and logical integrity of the system's smallest building blocks.

Integration Testing

Integration testing verifies that different modules, components, and external services interact and communicate correctly. This level ensures that the designed architecture works as a cohesive system.

- **Objective:** To validate the flow of data across system boundaries.
- **Application Focus:**
 - **Service Layer Validation:** Ensuring that user interface components correctly interface with the Firebase Authentication service and the Firestore database.
 - **Atomic Transactions:** Critically testing complex, sequential operations like the **Appointment Reschedule** feature. This involves confirming the reliable, non-corrupted sequencing of database calls required to simultaneously free up the original slot and book the new one.

9.2 Performance / Load Testing

As the HMS is a multi-user system where timely service is critical (a non-functional requirement), performance testing is mandatory to guarantee system responsiveness and stability under anticipated usage peaks.

- **Objective:** To measure the system's stability, responsiveness, and resource utilization efficiency under various load conditions.

- **Load Test Focus (Critical NFR):** Simulating high-concurrency scenarios for the **Appointment Booking** workflow. This is essential to guarantee that the system can handle numerous patients attempting to access and modify the same resource data simultaneously without experiencing deadlocks, data corruption, or service degradation.
- **Performance Metrics:** Measuring the time required for critical data retrieval operations (e.g., listing clinics, fetching appointment availability).
- **Acceptance Criteria:** The system must maintain an average response time of under **3 seconds** for all critical, state-changing transactions (booking/rescheduling) and under **1.5 seconds** for static data loads, even when tested under peak simulated load conditions.

9.3 Security Testing

Given the system's responsibility for handling sensitive health and personal information, security testing addresses the paramount non-functional requirement of data confidentiality and integrity.

- **Authentication & Authorization:** Systematically testing the robustness of the **Firestore Security Rules**. This verification ensures strict adherence to the principle of least privilege, confirming that patients can only read or write to their own records and that unauthorized users cannot access any sensitive clinical or administrative data.
- **Vulnerability Testing:** Utilizing techniques derived from **Decision Table Testing** to deliberately inject malformed or malicious data (e.g., boundary condition overloads, basic injection strings) to confirm that both client-side input validation and back-end rules successfully reject the input without compromising system stability or security.
- **Session Management:** Verifying the integrity of user sessions, ensuring tokens are properly validated, expire securely, and cannot be hijacked or reused by unauthorized entities.

9.4 User Acceptance / UI Testing

This final phase confirms that the application satisfies the stated functional requirements and delivers a user-friendly experience that aligns with the initial user stories.

- **Technique: Decision Table Testing (Input Validation):** This rigorous technique was applied to the Sign-Up and Login forms. It ensures complete coverage of all logical paths by systematically testing the success condition alongside all necessary failure conditions (e.g., weak password, invalid email format, missing required fields), guaranteeing robust error handling.
- **Technique: Use Case Testing (Workflow Validation):** This approach validates complex, goal-oriented sequences of user actions, specifically the Appointment life cycle (Booking, Rescheduling, Cancelling). This testing confirms that the entire end-to-end

process, from navigation and interaction to final database confirmation is completed successfully according to the patient/ user's perspective.

- **UI/UX Validation:** Ensuring the application's mobile interface is fully responsive across different devices, adheres to accessibility standards, and provides clear, immediate feedback for all interactions.

10. Appendices

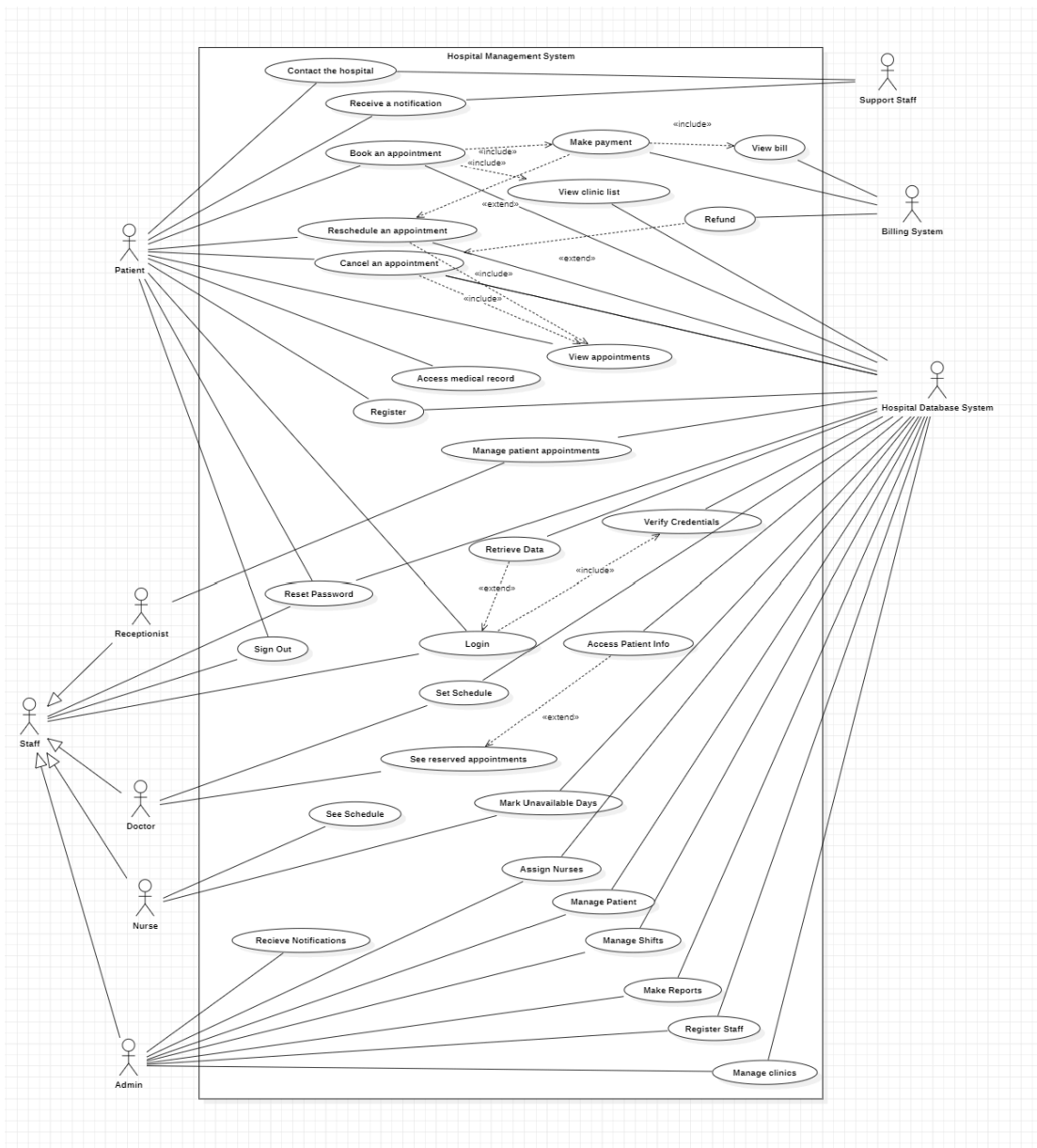
- **Appendix A: Glossary**

Term	Definition
Patient	A user who books and attends appointments in Carevo.
Appointment	A scheduled visit between a patient and a doctor at a specific time and clinic.
Shift	A work period assigned to a nurse or doctor.
Support Group	Staff responsible for handling patient messages and feedback.
API	Used in sequence diagrams (Booking API, Billing API, Scheduling API).
Authentication / Authorization	Used in the Authentication module description and security sections.
Backend / Frontend	Used when explaining client-server architecture and React Native + Firebase.
Cloud Firestore / Firebase / Firebase Authentication	Referenced in architecture, data, and risk sections.
React Native	Explicitly used for the mobile app implementation
Availability Status	Used in appointment and doctor-schedule design
Messaging / Support Portal	Used by support group members to read and reply to patient messages stored by the Communication and Support module
Doctor Availability / Schedule	Used by doctors to indicate when they are available for patient bookings, used as the source for patient-visible appointment options.
Boundary (UI) Class	A design-level class representing user interface screens and interactions (login form, appointment form, schedule view) that mediate between users and controller/domain classes
Controller Class	A design-level class such as Admin that coordinates requests between UI and

	domain entities, encapsulating higher-level workflows like assigning shifts or generating reports
Module	A logical grouping of related functionality in the design
Notification Service	Based on Firebase Cloud Messaging, responsible for sending push notifications

• Appendix B: Additional Diagrams

Use cases diagram:



- **Appendix C: External Services / Libraries**

Service / Library	Purpose	Notes
Firebase Authentication	Sign up, login, logout, forget password	Email/password auth, verification emails.
Cloud firestore	Data storage for users, clinics, appointments, messages	Real-time sync, offline cache.
React Native + Expo	Mobile app framework	Runs on Android/iOS devices.
React navigation	Screen navigation flows	Login → Dashboard → Clinics → Appointment.